

datalab 报告

姓名：王思童

学号：2025201696

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_
37	1	1	2	4	4	3	3	4	4	4	3

test 截图：



解题报告

亮点

- 1. float64_f2i
- 2. reverse
- 3. leftBitCount
- 4. floatPower2
- 5. logtwo

bitAnd

```
int bitAnd(int x, int y) {  
    return ~(~x | ~y);  
}
```

思路：
根据德摩根定律，两个变量的按位与，等价于两个变量分别取反后再按位或的整体取反。

bitXor

```
int bitXor(int x, int y) {  
    return ~(~x & ~y) & ~(x & y);  
}
```

思路：
异或是对应位不同则结果为1，相同则结果为0，可表示为 $(x \mid y) \& \sim(x \& y)$ ，再利用德摩根定律将 $x \mid y$ 转换为 $\sim(x \& \sim y)$ 。

samesign

```
int sameSign(int x, int y) {  
    if (!x && !y)  
    {  
        // 都等于0  
        return 1;  
    }  
    else if (!(x && y))  
    {  
        // 其中一个等于0  
        return 0;  
    }  
    else  
    {  
        // 都不等于0  
        return !((x ^ y) >> 31);  
    }  
}
```

思路：

0符号位为0但不是正数，需要单独判断。若!x && !y，两数均为0，则返回1；若!(x && y)，两数中有一个为0，则返回0；其余情况则需判断符号位是否相同，若相同则返回1，不同则返回0，所以用异或后右移31位取出符号位再取反。

logtwo

```
int logtwo(int v) {
    int res = 0;

    // 分成两个16位判断是否有1
    res = ((v >> 16) > 0) << 4; // 16位以上有1, res=16; 没有1, res=0

    // 右移res位后, 1一定在低16位, 分成两个8位判断是否有1
    res |= (((v >> res) >> 8) > 0) << 3; // 8位以上有1, res+=8; 没有1, res=0

    // 分成两个4位判断是否有1
    res |= (((v >> res) >> 4) > 0) << 2; // 4位以上有1, res+=4; 没有1, res=0

    // 分成两个2位判断是否有1
    res |= (((v >> res) >> 2) > 0) << 1; // 2位以上有1, res+=2; 没有1, res=0

    // 最后检查第1位
    res |= (((v >> res) >> 1) > 0); // 1位以上有1, res+=1; 没有1, res=0

    return res;
}
```

思路：

采用二分移位法实现，每次判断1是否在剩余有效位的高半区，若在则将对半区的位数加到结果当中，并将搜索范围搜索到高半区；若不在则保留在低半区。

16位区间判断：将v右移16位，若结果大于0，则说明1在高16位，结果累加16（即1右移4位）；否则说明1在低16位，结果不变。

8位区间判断：将v右移res位，现在剩余16位需要判断，再将v右移8位，判断1是否在高8位，若在则res加8，不在则不变

4位区间判断：同理判断高4位，存在则res加4，不在则不变

2位区间判断：同理判断高2位，存在则res加2，不在则不变

1位区间判断：同理判断高1位，存在则res加1，不在则不变

byteSwap

```
int byteSwap(int x, int n, int m) {
    int shift_n = n << 3; // n字节左移3位(x8)，得到n字节的位移量
    int shift_m = m << 3; // m字节左移3位(x8)，得到m字节的位移量

    int byte_n = (x >> shift_n) & 0xFF; // 提取第n字节
    int byte_m = (x >> shift_m) & 0xFF; // 提取第m字节

    // 清除第n字节和第m字节
    int mask = ~((0xFF << shift_n) | (0xFF << shift_m));
    int res = x & mask;

    // 将第n字节和第m字节交换
    res |= (byte_n << shift_m) | (byte_m << shift_n);
    return res;
}
```

思路：采用“提取->清空->写回”的操作思路

1. 将n,m左移3位，得到对应二进制位偏移量
2. 通过移位配合掩码，提取两个目标字节
3. 构造掩码清空原数中两个字节
4. 将两个提取的字节交换位置写回

reverse

```
unsigned reverse(unsigned v) {  
    // 奇数位和偶数位交换  
    v = ((v >> 1) & 0x55555555) | ((v & 0x55555555) << 1);  
    // 2位一组交换  
    v = ((v >> 2) & 0x33333333) | ((v & 0x33333333) << 2);  
    // 4位一组交换  
    v = ((v >> 4) & 0x0F0F0F0F) | ((v & 0x0F0F0F0F) << 4);  
    // 8位一组交换  
    v = ((v >> 8) & 0x00FF00FF) | ((v & 0x00FF00FF) << 8);  
    // 16位一组交换  
    v = (v >> 16) | (v << 16);  
    return v;  
}
```

思路：
采用分治思想，通过5轮分组交换，依次以1位、2位、4位、8位16位为单位，配合掩码提取每组高低半部分并交换位置，实现整体反转。

logicalShift

```
int logicalShift(int x, int n) {  
    // >> 是算数右移  
    int res = (x >> n);  
    int mask = ~(((1 << 31) & (!!n << 31)) >> (n + (~0))); // 如果n=0则不用移位  
    res &= mask;  
    return res;  
}
```

思路：先对有符号数进行算术右移，再构造高n位为0、其余位为1的掩码清除高位符号扩展位，同时通过!!n兼容n=0的情况，保证移位0位时结果不变。

leftBitCount

```
int leftBitCount(int x) {
    int cnt = 0;
    int mask;

    // 检查高16位是否全1
    mask = !~(x >> 16); // 全1 mask=1; 有0 mask=0
    cnt += (mask << 4); // 全1 +16; 全0 +0
    x <<= (mask << 4); // x左移, 把统计过的1去掉

    // 检查高8位
    mask = !~(x >> 24);
    cnt += (mask << 3);
    x <<= (mask << 3);

    // 检查高4位
    mask = !~(x >> 28);
    cnt += (mask << 2);
    x <<= (mask << 2);

    // 检查高2位
    mask = !~(x >> 30);
    cnt += (mask << 1);
    x <<= (mask << 1);

    // 检查最高位
    mask = !~(x >> 31);
    cnt += mask;
    x <<= mask;

    mask = !~(x >> 31);
    cnt += mask;

    return cnt;
}
```

思路：采用二分分组法统计左端连续 1 的个数，替代逐位遍历以压缩运算次数。从高位到低位依次以 16 位、8 位、4 位、2 位为单位判断高位是否全为 1：若全 1 则累加对应位数，并将 x 左移丢弃已统计的部分，继续检查剩余低位；2 位分组处理完成后，剩余未检查的高位最多还有 2 位，因此通过两次右移 31 位做单比特校验，依次检查剩余的每一位，确保覆盖所有连续 1 的场景。

float_i2f

```
unsigned float_i2f(int x) {
    unsigned sign = (x >> 31) & 1; // 符号位
    unsigned ux = x;
    if (sign) {
        ux = -ux; // 取绝对值
    }

    if (ux == 0) {
        return 0;
    }
    unsigned temp = ux;
    unsigned exp = 0; // 指数位
    while (temp >> 1) {
        temp >>= 1;
        exp++;
    }

    unsigned n = ux << (32 + ~exp); // 将最高位移到第31位，要移动 31-exp 位
    unsigned f = (n >> 8) & 0x7FFFFF; // 尾数位，取23位
    unsigned low = n & 0xFF; // 低8位，用于舍入
    unsigned up = (low > 0x80) | ((low == 0x80) & (f & 1)); // 舍入判断
    f += up; // 尾数加上舍入位
    if (f & 0x800000) { // f = 0x7FFFFFFF且需要向上舍入，导致尾数溢出
        f = 0;
        exp++;
    }
    return (sign << 31) | ((exp + 127) << 23) | f;
}
```

思路：

1. 提取符号位，对整数取绝对值，特殊值0直接返回0
2. 移位遍历找到最高有效位，得到原始指数
3. 将整数左移使最高位在第31位，截取高23位作为尾数，低8位用于舍入判断
4. 按照银行家舍入规则，若导致尾数溢出则指数+1、尾数清零
5. 指数加上偏移量127得到阶码，拼接符号位、阶码、尾数得到最终结果

floatScale2

```
unsigned floatScale2(unsigned uf) {
    unsigned sign = uf & 0x80000000; // 符号位
    unsigned exp = (uf >> 23) & 0xFF; // 指数
    unsigned frac = uf & 0x7FFFFFFF; // 尾数

    // 1. 阶码全1, 返回原数
    if (exp == 0xFF) {
        return uf;
    }

    // 2. 阶码为0, 非规格化数, 将尾数左移一位
    if (exp == 0) {
        frac <<= 1; // 左移1位 数值*2
        if (frac & 0x80000000) { // 如果左移后尾数溢出, 变为规格化数
            exp = 1; // 阶码变为1
            frac &= 0x7FFFFFFF; // 清除溢出的位
        }
    }

    // 3. 阶码不为0, 规格化数, 阶码加1
    else {
        exp += 1;
        if (exp == 0xFF) { // 如果阶码溢出, 变为无穷大
            frac = 0; // 尾数清零
        }
    }

    return sign | (exp << 23) | frac;
}
```

思路:

1. 特殊值处理: 若阶码全1, 返回原数
2. 非规格化数处理: 若阶码全0, 左移一位数值乘2, 若尾数最高位为1, 则乘2后变为规格化数, 阶码为1, 尾数保留低23位
3. 规格化数处理: 阶码加1实现乘2, 若阶码加1后变为全1, 则变为无穷大, 尾数清零

float64_f2i

```
int float64_f2i(unsigned uf1, unsigned uf2) {
    unsigned sign = (uf2 >> 31) & 1; // 符号位
    unsigned exp = (uf2 >> 20) & 0x7FF; // 阶码
    unsigned frac_high = uf2 & 0xFFFFF; // 尾数高20位
    unsigned frac_low = uf1; // 尾数低32位
    int real_exp = exp - 1023; // 真实指数，偏移量1023

    // 阶码全1 0x7FFF
    if (exp > 0x7FE)
    {
        return 0x80000000;
    }

    // 真实指数小于0，舍入为0
    if (real_exp < 0)
    {
        return 0;
    }

    // 真实指数>=31
    if (real_exp >= 31)
    {
        return 0x80000000;
    }

    unsigned hi32 = (1 << 20) | frac_high; // 还原最高位的1
    int shift = 52 - real_exp; // 需要右移的总位数
    unsigned abs_val;

    if (shift >= 32)
    {
        abs_val = hi32 >> (shift - 32);
    }
    else
    {
        abs_val = (hi32 << (32 - shift)) | (frac_low >> shift);
    }

    if (!sign)
    {
        return abs_val;
    }
    else
    {
        return -abs_val;
    }
}
```

思路：

1. 从输入的高低32位中分别提取符号位、11位阶码及52位尾数的高20位和低32位
2. 阶码全1，直接返回0x80000000；真实指数小于0，舍入为0；真实指数>=31，超出32位有符号整数表示范围，返回0x80000000
3. 补上规格化数隐含的最高位1，根据真实指数计算右移总位数，通过右移丢弃小数部分实现向零舍入；分移位置是否超过32位两种情况，拼接高低位尾数得到32位整数绝对值
4. 根据符号位对绝对值进行处理，得到最终有符号整数值

floatPower2

```
unsigned floatPower2(int x) {
    // 符号位一定是0, 可以不用
    unsigned exp = 0;
    unsigned frac = 0;

    if (x > 127)
    {
        exp = 0xFF;
    }
    else if (x >= -149 && x <= -127)
    {
        // 非规格数
        exp = 0;
        int shift = 126 + 23 + x;
        frac = 1 << shift;
    }
    else if (x >= -126 && x <= 127)
    {
        // 规格数
        exp = x + 127;
    }
    else
    {
        // 数太小, 下溢
        return 0;
    }

    return (exp << 23) | frac;
}
```

思路：

1. 上溢处理：当 $x > 127$ 时，超出规格化数的最大指数范围，阶码全1
2. 非规格化数处理：当 $-149 \leq x \leq -127$ 时，阶码全0，将1左移对应位数放入尾数字段
3. 规格化数处理：当 $-126 \leq x \leq 127$ 时，为正常规格化数，阶码为真实指数 x 加上偏移量 127，尾数恒为 0

反馈/收获/感悟/总结

这个lab的难度前几个函数比较简单，而后面的函数对我来说难度不小，尤其是浮点数部分，我认为原因是我对于浮点数以及仅通过位运算操作数字的方法还不太熟练，我通过询问大模型以及PPT上的例题已经得到了很好的理解。我发现在看到一道题的时候我可能没办法很快想到解法，我通过纸笔演算等方法能看懂大模型给出的解答，但是完全自己想的话就很难。

参考的重要资料

PPT2_编码上中的分治掩码数1的例题对于reverse函数有很大帮助